

IMPERIAL

IMPERIAL COLLEGE LONDON

IMPERIAL-X

Adaptive Resource Placement for Cyber Defense: A Multi-Agent Reinforcement Learning Approach

Author:

Miracle Akanmode

Supervisor:

Kelly Zhang

Submitted in partial fulfillment of the requirements for the MSc degree
in
Artificial Intelligence Applications and Innovation of Imperial College
London

September 11, 2025

Contents

1	Problem Formulation and Environment	iii
1.1	Decision-making problem formulation	iii
1.2	Network Representation and State Space	iii
1.2.1	Mathematical Foundation	iv
1.2.2	Implementation Details	iv
1.3	Red Agent (Fixed Adversary)	v
1.3.1	Attack Behavior	v
1.3.2	Role in Defensive Training	v
1.3.3	State Space and Actions	v
1.4	Blue Agent (Learnable Defender)	vi
1.4.1	State Space and Observations	vi
1.4.2	Action Space and Defense Operations	vii
1.5	Reward System: Mathematical Relationships and Timing	vii
1.5.1	Environment-Agent Interaction Dynamics	x
1.6	Environment Selection and Extensibility Rationale	x
1.6.1	Cyberwheel Configuration System Architecture	xi
1.6.2	Unified Mathematical Notation	xi
1.7	Configuration–Mathematical Parameter Mapping	xiii

Abstract

Modern cybersecurity defense faces an asymmetric challenge: sophisticated adversaries adapt tactics faster than traditional rule-based systems can respond. We formulate cyber defense as a multi-agent reinforcement learning problem where defensive agents learn adaptive deception strategies through adversarial interaction. Using Proximal Policy Optimization, our approach trains agents to strategically deploy honeypots and decoy systems that mislead attackers while protecting critical assets. The key insight is that learned adaptive placement creates strategic uncertainty for attackers, disrupting reconnaissance and lateral movement phases. We validate this approach through comprehensive evaluation against traditional baselines across diverse network configurations, integrating real MITRE ATT&CK commands to ensure behavioral fidelity. Results demonstrate that PPO-based defensive strategies achieve superior performance compared to static and rule-based approaches while maintaining the consistency and reliability required for enterprise deployment.

1 Problem Formulation and Environment

1.1 Decision-making problem formulation

We formulate the cyber defense problem as an episodic reinforcement learning environment where each episode represents a complete cyber attack scenario. Episodes have finite length T_{max} , representing the maximum number of decision steps before environment reset. We use T to denote the total number of training episodes.

The red and blue agents operate with distinct but interacting state representations and action spaces, reflecting their asymmetric roles in cybersecurity. The red agent uses internal state tracking $\mathcal{S}^{(r)}$ while the blue agent uses an explicit state vector $\mathcal{S}^{(b)} \subset \mathbb{R}^{d_b}$. Similarly, $\mathcal{A}^{(r)}$ and $\mathcal{A}^{(b)}$ represent their respective action spaces.

The agents operate in a turn-based fashion within each timestep, modeling realistic attack-defense dynamics. At each timestep $t \in [1 : H]$ within an episode, the red agent observes the network state and executes an attack action first, followed by the blue agent observing resulting alerts and taking defensive action.

At each timestep t within an episode, the red agent updates its internal state tracking and the blue agent observes state vector $S_t^{(b)} \in \mathcal{S}^{(b)}$, then both agents select actions $A_t^{(r)} \in \mathcal{A}^{(r)}$ and $A_t^{(b)} \in \mathcal{A}^{(b)}$ according to policies $\pi_{\text{fixed}}^{(r)}$ and $\pi^{(b)}$.

The environment provides immediate rewards $R_t^{(r)}$ and $R_t^{(b)}$ based on action outcomes and network state. These rewards exhibit adversarial properties: successful red attacks on real assets provide negative blue rewards, while successful deception (red attacking decoys) provides positive blue rewards.

The red agent operates as a fixed adversary with deterministic policy $\pi_{\text{fixed}}^{(r)}$, requiring no optimization objective.

The blue agent’s learning objective is to maximize the expected return:

$$J^{(b)}(\pi^{(b)}) = \mathbb{E}_{\pi^{(b)}, \pi_{\text{fixed}}^{(r)}} \left[\sum_{t=0}^{H-1} \gamma^t R_t^{(b)} \right] \quad (1)$$

where:

- $\gamma = 0.99$ is the discount factor
- H is the episode length
- $R_t^{(b)}$ is the unified blue reward function defined above
- The expectation is taken over trajectories generated by blue policy $\pi^{(b)}$ interacting with fixed red policy $\pi_{\text{fixed}}^{(r)}$

Important Note on Adversary Model This research focuses on training blue defensive agents against a **fixed, pre-trained red adversary**. We do not train or modify the red agent—it serves as a consistent adversarial opponent that follows MITRE ATT&CK framework patterns. This design choice ensures reproducible evaluation and isolates the learning dynamics to the defensive agent only.

1.2 Network Representation and State Space

The environment models enterprise network topologies with realistic characteristics as shown in Figure 1. Networks consist of multiple subnets (demilitarized zone [DMZ], internal networks) containing a mix of real assets and strategically placed decoys.

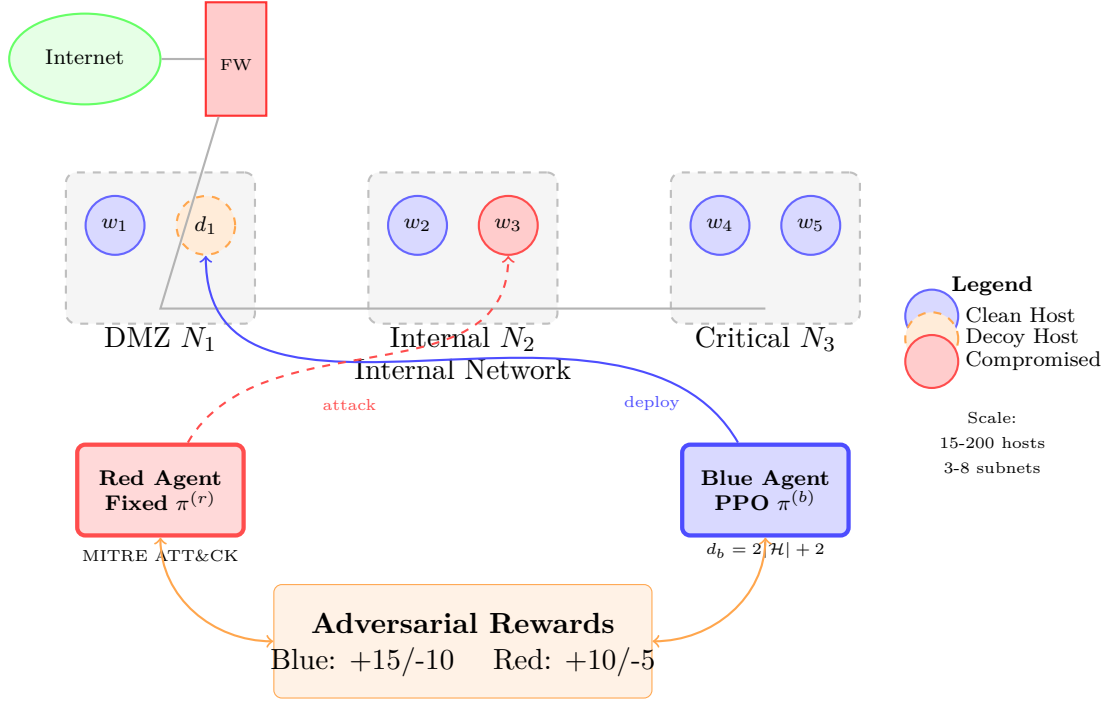


Figure 1: Multi-agent cyber defense environment showing DMZ and internal subnets containing real assets and decoys. Red agent executes attacks with MITRE ATT&CK (dashed red arrows) while Blue agent learns optimal decoy deployment with PPO (solid blue arrows). Rewards provide feedback for learning.

1.2.1 Mathematical Foundation

The network environment is represented as a directed graph $G = (V, E)$:

$$G = (V, E) \text{ where } G = \text{nx.DiGraph}() \quad (2)$$

$$V = \mathcal{W} \cup \mathcal{N} \cup \mathcal{R}_{net} \quad (3)$$

$$E \subseteq V \times V \quad (4)$$

where $\mathcal{W}$ represents hosts (computers/devices), $\mathcal{N}$ represents subnets, $\mathcal{R}_{net}$ represents routers, and E represents directed network connections.

Each host $w_i \in \mathcal{W}$ is characterized by a comprehensive state vector:

$$w_i = \langle \text{IP}_i, \text{OS}_i, \text{Services}_i, \text{Vulns}_i, \text{is_compromised}_i, \text{decoy}_i \rangle \quad (5)$$

where IP_i is the IP address, OS_i is the operating system, Services_i are running services, Vulns_i are vulnerabilities, $\text{is_compromised}_i \in \{0, 1\}$ indicates compromise status, and $\text{decoy}_i \in \{0, 1\}$ indicates whether the host is a honeypot.

1.2.2 Implementation Details

The environment is implemented with these details. The architecture uses `nx.DiGraph` directed graph structure with scale support for larger networks, the val-

idated empirical range for this research covers 15-200 hosts. MITRE integration implements 7 kill-chain phase techniques following the ATT&CK methodology through YAML-driven modularity across all components.

Having established the network environment representation, we now detail the specific agent implementations operating within this framework, beginning with the adversarial red agent.

1.3 Red Agent (Fixed Adversary)

The red agent serves as a fixed, pre-trained cyber adversary implementing MITRE ATT&CK framework patterns. It provides consistent adversarial pressure essential for reproducible defensive training and evaluation.

1.3.1 Attack Behavior

The red agent executes a structured kill chain that progresses through discovery via network scanning and reconnaissance, followed by exploitation to compromise vulnerable hosts. Once established, the agent performs lateral movement to spread throughout the network from its initial foothold, ultimately attempting to achieve impact on target systems. This deterministic progression ensures reproducible attack scenarios that enable fair comparison of defensive strategies across experiments.

1.3.2 Role in Defensive Training

The fixed adversary design provides essential benefits for rigorous experimental evaluation. Standardized attack patterns enable controlled comparison of blue agent performance without the confounding variables introduced by adaptive opponents. The MITRE ATT&CK-based progression reflects real-world attack sophistication while maintaining experimental reproducibility. Most importantly, the fixed opponent eliminates co-evolution complexity, allowing us to isolate defensive learning dynamics and focus specifically on blue agent adaptation without concurrent adversarial learning.

1.3.3 State Space and Actions

The red agent maintains situational awareness through structured internal state tracking rather than a single observation vector. At each timestep t , the agent’s knowledge consists of its current position $\text{current_host}_t \in \mathcal{W}$, accumulated network intelligence through history_t , and detailed reconnaissance data about discovered hosts in $\text{tracked_hosts}_t \subseteq \mathcal{W}$. This internal state representation can be formalized as:

$$\mathcal{S}_t^{(r)} = \{\text{current_host}_t, \text{history}_t, \text{tracked_hosts}_t, \text{services_map}_t, \text{unimpacted_hosts}_t\} \quad (6)$$

The agent’s operational capabilities expand dynamically as network knowledge grows. Each discovered host becomes a potential target for the seven-phase MITRE ATT&CK kill chain $\mathcal{K} = \{\text{pingsweep}, \text{portscan}, \text{discovery}, \text{lateral-movement}, \text{privilege-escalation}, \text{impact}\}$. The action space at timestep t reflects this reconnaissance-driven expansion:

$$\mathcal{A}_t^{(r)} = \mathcal{K} \times \text{tracked_hosts}_t \quad \text{with size} \quad |\mathcal{A}_t^{(r)}| = 7 \times |\text{tracked_hosts}_t| \quad (7)$$

This formulation captures the adaptive nature of cyber attacks, where an adversary’s capabilities grow through successful reconnaissance and network mapping activities.

The adversarial learning framework is illustrated in Figure 2.

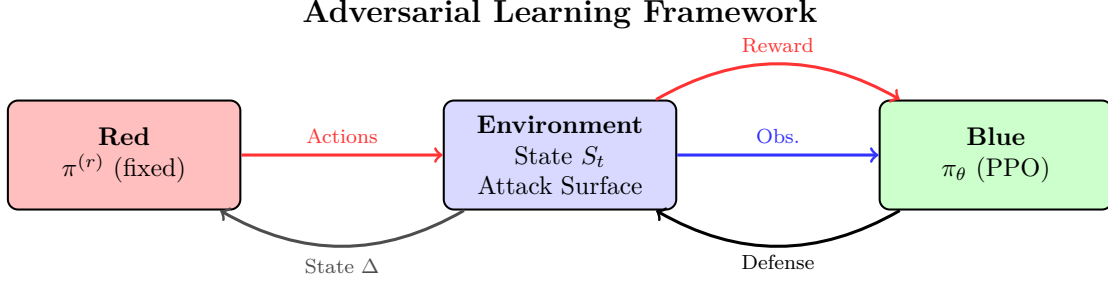


Figure 2: Adversarial Learning Loop. The fixed red agent policy produces consistent attack actions, the environment state gets updated, and the blue agent receives observations and rewards, and learns adaptive defensive strategies through PPO. The environment facilitates this adversarial interaction and learning signal generation

1.4 Blue Agent (Learnable Defender)

The blue agent implements adaptive defensive strategies emphasizing cyber deception and strategic network isolation through PPO-based reinforcement learning.

1.4.1 State Space and Observations

The blue agent’s state space $S_t^{(b)} \in \mathbb{R}^{d_b}$ has a simplified observation structure:

$$S_t^{(b)} = \begin{bmatrix} \text{current_alerts}_t \\ \text{alert_history}_t \\ \text{metadata}_t \end{bmatrix} \quad (8)$$

where $\text{current_alerts}_t \in \{0, 1\}^{|\mathcal{W}|}$ represents immediate threat indicators per host, $\text{alert_history}_t \in \{0, 1\}^{|\mathcal{W}|}$ captures cumulative attack memory enabling pattern learning, and $\text{metadata}_t \in \mathbb{R}^2$ contains environmental constants: a fixed value of -1 and the total number of active decoys.

State Space Dimensionality Analysis

The total dimensionality is $d_b = 2|\mathcal{W}| + 2$, comprising alert components with $2|\mathcal{W}|$ dimensions (current + historical per host) and metadata components with 2 dimensions (environmental constants: fixed value -1 and total decoy count).

This structure enables both immediate threat response and strategic pattern learning. Figure 3 provides a detailed breakdown of the observation vector structure and dimensionality.

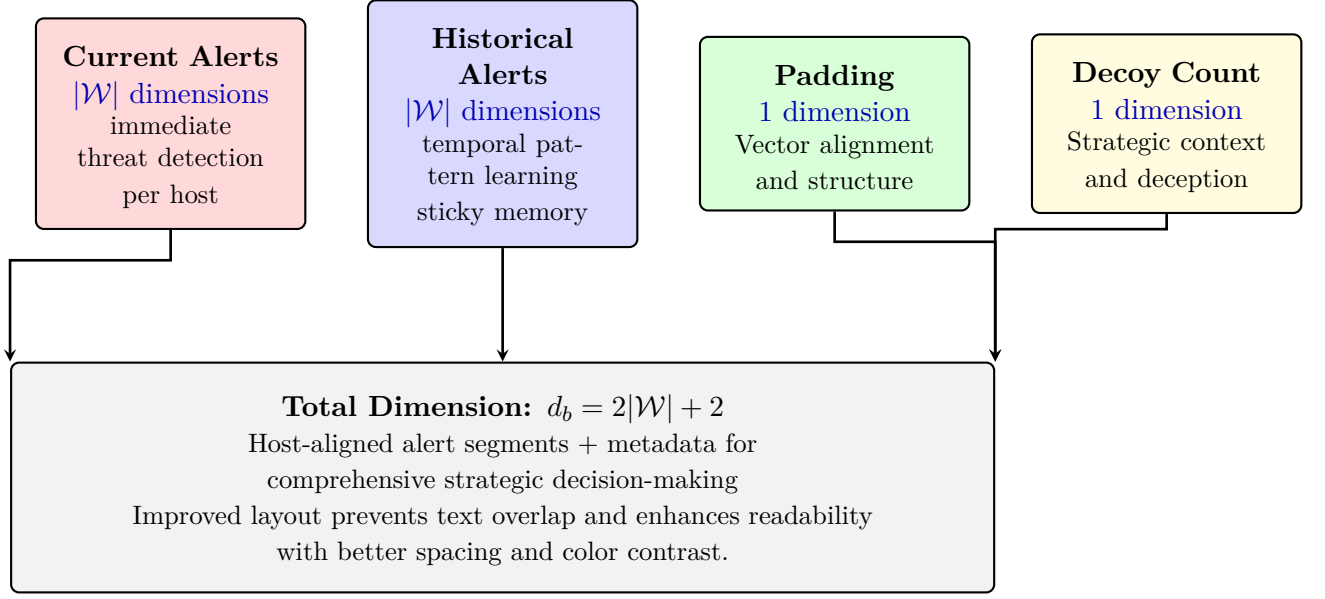


Figure 3: Blue Observation Vector Structure: Comprehensive state representation with current alerts, historical patterns, padding for alignment, and decoy count for strategic context. Improved layout prevents text overlap and enhances readability with better spacing and color contrast.

1.4.2 Action Space and Defense Operations

The blue agent’s action space consists of defensive operations implemented as a discrete action space:

$$\mathcal{A}^{(b)} = \mathcal{A}_{\text{deploy}} \cup \mathcal{A}_{\text{remove}} \cup \{\text{nothing}\} \quad (9)$$

where $\mathcal{A}_{\text{deploy}} = \{(\text{deploy_decoy}, n_j) : n_j \in \mathcal{N}\}$ deploys a decoy on subnet n_j , $\mathcal{A}_{\text{remove}} = \{(\text{remove_decoy}, n_j) : n_j \in \mathcal{N}\}$ removes a decoy from subnet n_j , and nothing represents no action (observation only).

Total action space size: $|\mathcal{A}^{(b)}| = 2|\mathcal{N}| + 1$.

1.5 Reward System: Mathematical Relationships and Timing

This section establishes unified notation and explicit mathematical relationships between red and blue rewards.

Fundamental Reward Structure

Our system implements a **competitive reward structure** where red and blue agents have opposing objectives, formalized as:

$$R_t^{(b)} + R_t^{(r)} = C - \text{system_costs}_t \quad (10)$$

where C is a normalization constant and system costs represent computational overhead. This creates a near-zero-sum game where blue agent success directly corresponds to red agent failure.

Red Agent Reward Components

The red agent reward function is determined by the specific action executed, with rewards configured per action type as shown in the actual implementation.

Blue Agent Reward Components

The blue agent reward function emphasizes deception effectiveness through a $10\times$ multiplier when red agents attack decoys, while imposing negative rewards when real hosts are compromised. Additional costs are applied for deployment and removal actions as configured.

Unified Mathematical Relationship

The competitive relationship is implemented through direct coupling of red and blue rewards based on action outcomes and target types, as described in the implemented reward functions above.

Shared Outcome Variables: Both reward functions depend on common environmental variables including $\mathbf{1}[\text{technique}_i \text{ successful}]$ used in both $R_t^{(r)}$ and $R_t^{(b)}$ calculations, $\Delta\text{assets_compromised}_t$ where direct red gain becomes direct blue loss, and $\mathbf{1}[\text{detected}_t]$ where red penalty corresponds to blue monitoring success.

Reward Timing and Detection Mechanisms

Detection Timing: Rewards are computed immediately following each agent action based on the resulting environment state. Detection occurs when red agent actions trigger alert thresholds in the monitoring system, blue agent observes attack signatures in current observation vector, or decoy interactions generate immediate deception reward signals.

Multi-step Effects: While immediate rewards capture direct action consequences, our reward structure also accounts for delayed effects through the temporal discount factor $\gamma = 0.99$, allowing agents to learn multi-step strategies where early actions enable later tactical advantages.

Detection Delay Modeling: In our current implementation, detection is modeled as immediate for computational tractability. However, the framework accommodates delayed detection through state history mechanisms where attack indicators persist across multiple time steps before triggering alerts.

Unified Mathematical Formulation with Specific Parameter Values

Building on the mathematical relationships above, we provide the specific instantiation used in our experiments:

Let $\mathcal{N}_t = \{w_1, w_2, \dots, h_n\}$ be the network state at time t , where each host w_i has status $\sigma_i \in \{\text{clean}, \text{compromised}, \text{decoy}\}$.

Define detection events:

$$D_t^{\text{decoy}} = \mathbb{I}[\text{red attacks decoy at time } t] \quad (11)$$

$$D_t^{\text{real}} = \mathbb{I}[\text{red attacks real host at time } t] \quad (12)$$

$$D_t^{\text{success}} = \mathbb{I}[\text{red successfully compromises new host at time } t] \quad (13)$$

Blue Agent Reward Function $R_t^{(b)}$ (Implemented):

$$R_t^{(b)} = \begin{cases} \text{red_reward} \times 10 & \text{if red attacks decoy successfully} \\ -\text{red_reward} & \text{if red attacks real host successfully} \\ \text{blue_action_reward} & \text{if blue action succeeds} \\ 0 & \text{otherwise} \end{cases} + \text{recurring_rewards} \quad (14)$$

where the specific reward values from configuration are: `deploy_decoy` immediate reward = -20.0 , `remove_decoy` immediate reward = -1.0 , `nothing` immediate reward = 0.0 , and `red_reward` values are determined by the red agent’s action configuration. The reward calculation emphasizes deception success ($10\times$ multiplier) over deployment costs.

Red Agent Reward Function $R_t^{(r)}$ (Fixed Adversary): Since the red agent operates as a fixed adversary with deterministic strategy, its reward is determined by the action executed:

$$R_t^{(r)} = \text{immediate_reward[action]} + \text{recurring_reward[action]} \quad (15)$$

where the specific reward values from configuration are: `pingsweep` = 1 , `portscan` = 2 , `discovery` = 5 , `lateral-movement` = 10 , `privilege-escalation` = 20 , `impact` = 50 , `nothing` = 0 . All recurring rewards are 0 .

Adversarial Reward Properties: The reward functions exhibit adversarial properties in key interaction scenarios based on actual implementation:

$$\text{When red attacks decoy: } R_t^{(b)} = +\text{red_reward} \times 10 \quad (16)$$

$$R_t^{(r)} = +\text{red_reward} \text{ (action-specific)} \quad (17)$$

$$\text{When red attacks real host: } R_t^{(b)} = -\text{red_reward} \quad (18)$$

$$R_t^{(r)} = +\text{red_reward} \text{ (action-specific)} \quad (19)$$

RL Optimization Objective: Since only the blue agent learns (red agent is fixed), the learning objective is:

$$J^{(b)} = \mathbb{E} \left[\sum_{t=1}^T \sum_{h=1}^H \gamma^{h-1} R_h^{(b)} \right] \quad (20)$$

where the blue agent optimizes this objective using PPO while the red agent follows a fixed deterministic strategy.

where $\gamma = 0.99$ is the discount factor, T is the number of trials, and H is the episode horizon.

1.5.1 Environment-Agent Interaction Dynamics

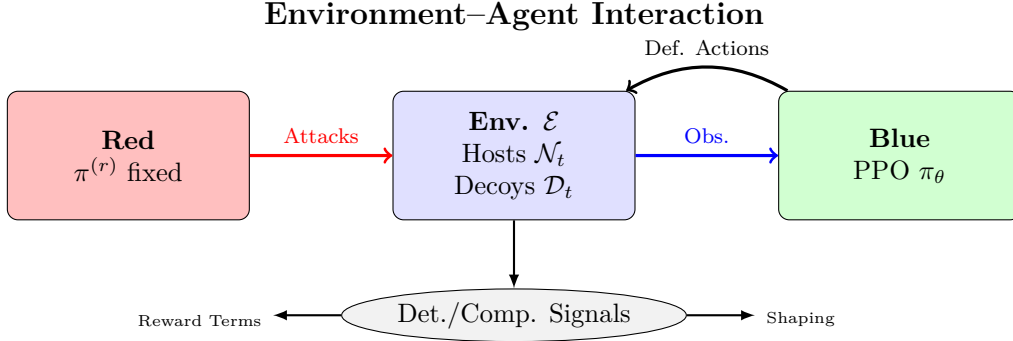


Figure 4: Tightened interaction loop: red produces attacks, environment updates host/decoy state, blue observes and deploys/removes decoys, signals feed reward.

1. **Action-Reward Causality:** Rewards are triggered by specific detection events within the same timestep as actions
2. **Detection Mechanism:** D_t^{decoy} and D_t^{success} are determined by red agent actions interacting with current network state $\mathcal{N}_t$
3. **Temporal Consistency:** All reward components $R_t^{(b)}$ are observed immediately at time t , eliminating credit assignment ambiguity
4. **Causality Chain:** Blue action at $t - 1$ modifies $\mathcal{N}_t \rightarrow$ Red action at t triggers detection $\rightarrow$ Rewards observed at t

Learning Signal Properties: This unified formulation provides PPO with consistent variables where all reward terms use the same mathematical framework and timing, clear adversarial structure through explicit zero-sum relationships in key interaction scenarios, temporal precision via exact specification of when rewards are observed relative to actions, and strategic incentives where $\alpha_2 \gg \alpha_1$ encourages strategic deception over reactive deployment.

1.6 Environment Selection and Extensibility Rationale

Why Cyberwheel? The Cyberwheel framework was developed by Oak Ridge National Laboratory (ORNL) as a modular, extensible platform for cybersecurity research and was selected after comparative review of alternative cyber-range simulation environments (e.g., CybORG, CAGE, OpenAI Gym network extensions) based on three key advantages: (1) **native deception support** for decoy deployment state transitions crucial to our research, (2) **modular YAML-driven configuration** enabling systematic experimentation and baseline comparison, and (3) **scalable architecture** supporting networks from 15 hosts to enterprise scale with deterministic adversary modes for controlled learning analysis.

ORNL Repository and Development Context: The original Cyberwheel framework and documentation are maintained by Oak Ridge National Laboratory, with development motivated by the need for reproducible, scalable cybersecurity RL research platforms. This aligns directly with our requirements for systematic baseline comparison and experimental rigor.

Design Considerations: Our experimental design focuses on establishing fundamental defensive learning behaviors using a fixed adversary model. This approach preserves interpretability of baseline defensive learning signals and enables systematic analysis of policy behavior without additional complexity from dynamic attacker adaptation, leveraging Cyberwheel’s modular design for controlled experimental conditions.

1.6.1 Cyberwheel Configuration System Architecture

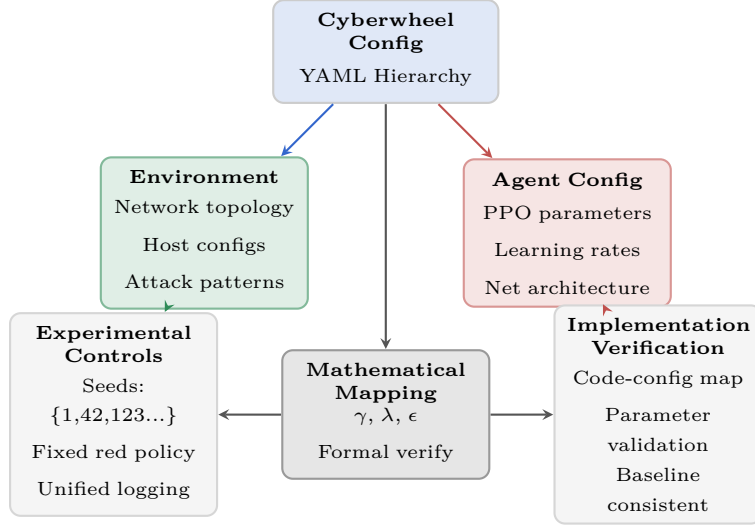


Figure 5: Cyberwheel Configuration System Architecture: YAML-driven modular design enables systematic experimental control through hierarchical parameter organization. Mathematical mapping ensures theoretical formulations align with implementation parameters, supporting reproducible comparative analysis.

1.6.2 Unified Mathematical Notation

We build on the problem formulation above to establish a comprehensive notation system used throughout this work:

Table 1: Unified Mathematical Notation Framework

Symbol	Domain	Definition
Network Environment		
$G = (V, E)$	Topology	Directed graph representing network topology
V	Vertices	Set of all graph vertices: $V = \mathcal{W} \cup \mathcal{N} \cup \mathcal{R}_{net}$
E	Edges	Set of directed network connections
$\mathcal{W} = \{w_1, \dots, w_n\}$	Hosts	Set of network hosts/devices
$\mathcal{N} = \{N_1, \dots, N_k\}$	Subnets	Set of network subnets
$\mathcal{R}_{net}$	Routers	Set of network routing devices
$\mathcal{D}$	Decoys	Set of available decoy types
$\mathcal{T}$	Techniques	Set of MITRE ATT&CK techniques (295 total)
Agent States & Actions		
$\mathcal{S}^{(r)}$	Red Agent	Red agent internal state tracking (no explicit vector)
$\mathcal{S}^{(b)} \subset \mathbb{R}^{d_b}$	Blue Agent	Blue agent state space
$d_b = 2 \mathcal{W} + 2$	Dimensions	Blue agent state space dimensionality
$\mathcal{A}^{(r)}$	Red Agent	Red agent action space (7 MITRE ATT&CK actions per known host)
$\mathcal{A}^{(b)}$	Blue Agent	Blue agent action space (deploy/remove decoys per subnet + nothing)
$S_t^{(r)}, S_t^{(b)}$	States	Agent states at timestep t
$A_t^{(r)}, A_t^{(b)}$	Actions	Agent actions at timestep t
Policies & Learning		
$\pi_{\text{fixed}}^{(r)}$	Red Agent	Fixed deterministic red policy
$\pi^{(b)}$	Blue Agent	Learnable blue policy (PPO)
$J^{(b)}(\pi^{(b)})$	Objective	Blue agent learning objective
H	Episodes	Maximum episode length (time steps)
T	Training	Total number of training episodes
t	Time	Current timestep within episode
γ	Learning	Discount factor (0.99)
Rewards		
$R_t^{(r)}, R_t^{(b)}$	Rewards	Immediate rewards at timestep t
$\mathcal{R}^{(r)}, \mathcal{R}^{(b)}$	Functions	Reward functions
Actual Reward Values (from configuration)		
Red Action Rewards	Values	pingsweep=1, portscan=2, discovery=5, lateral-movement=10, privilege-escalation=20, impact=50, nothing=0
Blue Action Rewards	Values	deploy_decoy=-20.0, remove_decoy=-1.0, nothing=0.0
Deception Multiplier	Blue	Red reward $\times 10$ when attacking decoys
Compromise Penalty	Blue	-Red reward when attacking real hosts

Having established the mathematical framework above, we now describe how this formulation maps to realistic network environments and the specific state space representations used by both agents.

1.7 Configuration–Mathematical Parameter Mapping

Table 2 links symbolic notation used in formal definitions to concrete configuration sources. Only parameters present in the current codebase are listed; unimplemented or future-planned elements are omitted.

Table 2: Mathematical Notation Framework: Mapping of symbols to configuration fields and implementation details

Symbol	Name	Config Source	Role
γ	Discount Factor	rl/ppo yaml	Temporal weighting of future rewards (set 0.99)
λ	GAE Parameter	rl/ppo yaml	Bias–variance trade-off in advantage estimation (set 0.95)
ϵ	PPO Clip Range	rl/ppo yaml	Trust region style constraint (set 0.2)
c_1	Entropy Coefficient	rl/ppo yaml	Encourages exploration (set 0.01)
c_2	Value Loss Coefficient	rl/ppo yaml	Balances policy vs critic optimization (set 0.5)
$ \mathcal{W} $	Host Count	topology yaml	Defines alert vector segments / state dimensionality
$ \mathcal{N} $	Subnet Count	topology yaml	Determines action mapping size (deploy/remove granularity)
H	Episode Horizon	env settings	Upper bound on per-episode decision steps
T	Training Episodes	run script args	Outer loop sample budget controlling convergence window
$\alpha_{1..4}$	Reward Weights	reward module	Shape deployment, deception, compromise, removal incentives
β_1, β_2	Red Reward Weights	reward module	Maintain adversarial structure (fixed opponent accounting)